Лучший курс по Python

int



Автор курса —
Никита Соболев

>_ Junior

Какие __магические__ методы превращают объект в int?

```python
class MyNumber:
    def __init__(self, number: int) -> None:
        self.number = number

    def __int__(self) -> int:
        return self.number


print(int(MyNumber(5)))  # 5
```

```python
class MyNumber:
    def __init__(self, number: int) -> None:
        self.number = number

    def __int__(self) -> int:
        return float(self.number)  # Error: float return!


print(int(MyNumber(5)))
# TypeError: __int__ returned non-int (type float)
```

```python
# typing.py

@runtime_checkable
class SupportsInt(Protocol):
    """An ABC with one abstract method __int__."""

    __slots__ = ()

    @abstractmethod
    def __int__(self) -> int:
        pass
```

```
>>> int('10')
10
>>> int('10', base=2)
```



Факт

```
>>> int('10')
10
>>> int('10', base=2)
2
```



Факт

```
>>> int('10')
10
>>> int('10', base=2)
2
>>> int('10', base=8)
8
```



Факт

```
>>> int('10')
10
>>> int('10', base=2)
2
>>> int('10', base=8)
8
>>> int('١٠', base=2)
```



Факт

```
>>> int('10')
10
>>> int('10', base=2)
2
>>> int('10', base=8)
8
>>> int('١٠', base=2)
2
```



Факт

__index__

__trunc__
__ceil__
__floor__

```
>>> import math

>>> math.ceil(4.5)
5
>>> math.ceil(-4.5)
-4

>>> math.floor(4.5)
4
>>> math.floor(-4.5)
-5

>>> math.trunc(4.5)
4
>>> math.trunc(-4.5)
-4
```

```
class ZeroItem:
    def __index__(self) -> int:
        return 0


letters = ['a', 'b', 'c']
print(letters[ZeroItem()])  # 'a'
print(bin(ZeroItem()))  # '0b0'
```

В Python2 было еще много других связанных методов:

__long__ __hex__

__oct__ __coerce__



Истори
я

Хитрые формы записи

```
>>> 0xa == 0b1010 == 0o12
True

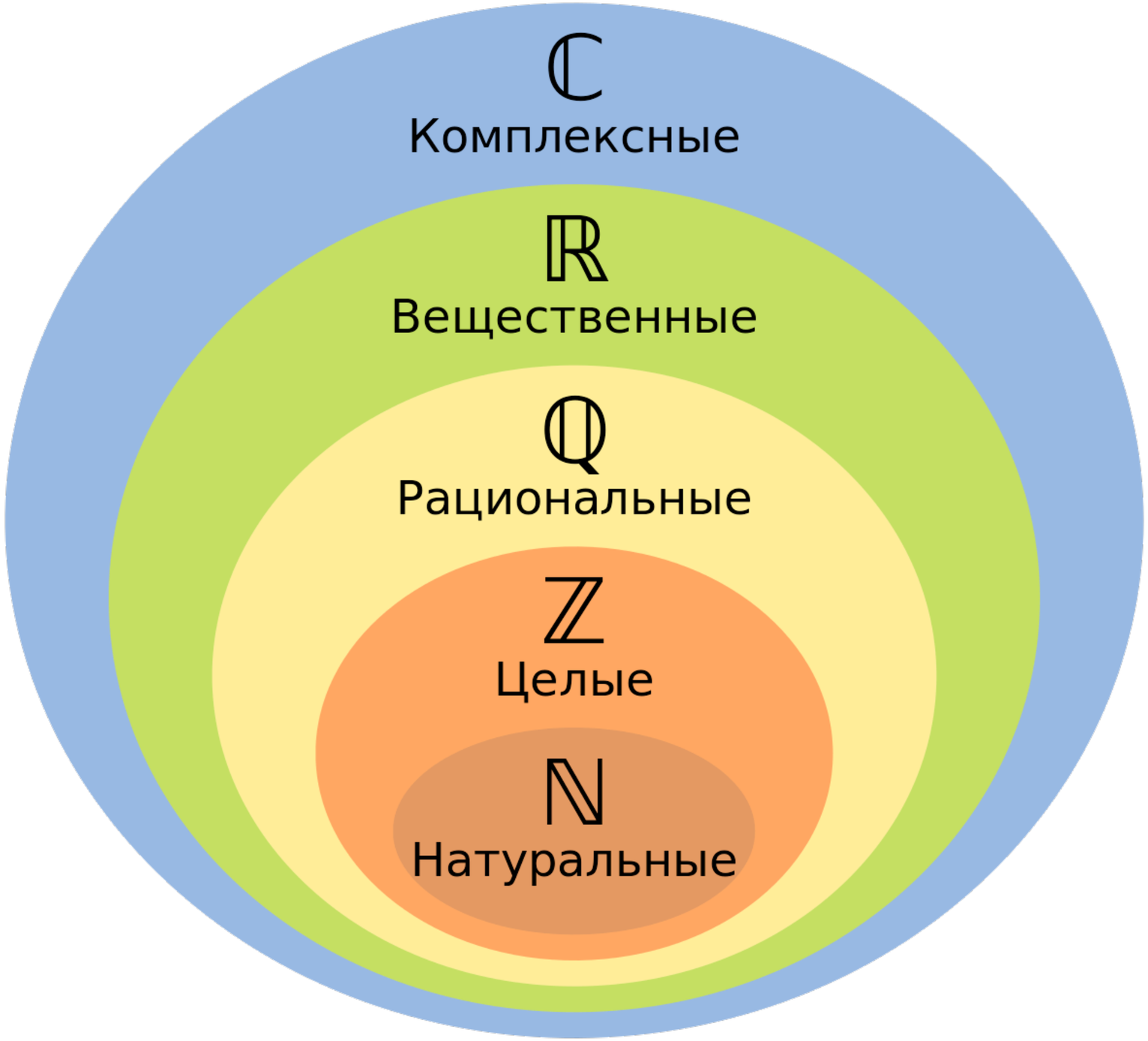
>>> 0xa_0 == 0X000000A0 == 1_60
True
```

```
>>> 0xa == 0b1010 == 0o12
True

>>> 0xa_0 == 0X000000A0 == 1_60
True

>>> oct(10)
'0o12'
>>> hex(10)
'0xa'
>>> bin(10)
'0b1010'
```

>_ Middle


ℂ
Комплексные
ℝ
Вещественные
ℚ
Рациональные
ℤ
Целые
ℕ
Натуральные

Number
:> Complex
:> Real
:> Rational
:> Integral

Абстрактные классы чисел или "Numeric Tower"



Демо

```
>>> from numbers import Integral

>>> issubclass(int, Integral)
True

>>> int.__bases__
(<class 'object'>,)
```

Для чего их используют?

> peps.python.org/pep-3141

> Для создания своих "числовых" типов

> Для стандартизации поведения чисел

Открытые проблемы

> Не стоит их использовать для аннотаций

> Не все числа все еще поддерживают все методы (`.register()` vs subclassing)

> Decimal

```
>>> from numbers import Complex
>>> isinstance(1, Complex)
True
```

```
>>> from numbers import Complex
>>> isinstance(1, Complex)
True

>>> Complex.__complex__
<function Complex.__complex__ at 0x10273f560>
```

```
>>> from numbers import Complex
>>> isinstance(1, Complex)
True

>>> Complex.__complex__
<function Complex.__complex__ at 0x10273f560>
>>> (1).__complex__()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'int' object has no attribute '__complex__'
```

>_ Senior

В Python2 был отдельный тип long, который теперь называется int



Истори
я

10L == 10



Истори
я

Следы все еще внутри

> PyLong_* CAPI

> longobject.c / test_long.py



Истори
я



Как устроен PyLong_Type?

```
static PyNumberMethods long_as_number = {
    (binaryfunc)long_add,        /*nb_add*/
    // ...
    long_long,                   /*nb_int*/
    long_float,                  /*nb_float*/
    long_long,                   /*nb_index*/
};
```

```c
PyTypeObject PyLong_Type = {
    PyVarObject_HEAD_INIT(&PyType_Type, 0)
    "int",                                      /* tp_name */
    offsetof(PyLongObject, long_value.ob_digit),  /* tp_basicsize */
    sizeof(digit),                              /* tp_itemsize */
    long_dealloc,                               /* tp_dealloc */
    long_to_decimal_string,                     /* tp_repr */
    &long_as_number,                            /* tp_as_number */
    (hashfunc)long_hash,                        /* tp_hash */
    Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE |
        Py_TPFLAGS_LONG_SUBCLASS |
        _Py_TPFLAGS_MATCH_SELF,                 /* tp_flags */
    long_doc,                                   /* tp_doc */
    long_richcompare,                           /* tp_richcompare */
    long_methods,                               /* tp_methods */
    long_getset,                                /* tp_getset */
    long_new,                                   /* tp_new */
    PyObject_Free,                              /* tp_free */
    .tp_vectorcall = long_vectorcall,
};
```

sys.maxsize

```
>>> import sys

>>> sys.maxsize
9223372036854775807
>>> sys.maxsize + 2
9223372036854775809
```

```
/* The absolute value of a number is equal to
     SUM(for i=0 through ndigits-1) ob_digit[i] * 2**(PyLong_SHIFT*i)

   The sign of the value is stored in the lower 2 bits of lv_tag.

    - 0: Positive
    - 1: Zero
    - 2: Negative

   The third lowest bit of lv_tag is reserved for an immortality flag, but is
   not currently used.

   In a normalized number, ob_digit[ndigits-1] (the most significant
   digit) is never zero.  Also, in all cases, for all valid i,
        0 <= ob_digit[i] <= PyLong_MASK.
*/
typedef struct _PyLongValue {
    uintptr_t lv_tag; /* Number of digits, sign and flags */
    digit ob_digit[1];
} _PyLongValue;

struct _longobject {
    PyObject_HEAD
    _PyLongValue long_value;
};
```



Py_ssize_t

peps.python.org/pep-0353

```
>>> import sys

>>> [].pop(sys.maxsize)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: pop from empty list
```

```
>>> import sys

>>> [].pop(sys.maxsize)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: pop from empty list

>>> [].pop(sys.maxsize + 1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OverflowError: Python int too large to convert to C ssize_t
```



```
/*[clinic input]
@critical_section
list.pop

    index: Py_ssize_t = -1
    /

[clinic start generated code]*/

static PyObject *
list_pop_impl(PyListObject *self, Py_ssize_t index)
/*[clinic end generated code: ... ]*/
{
  // ...
}
```

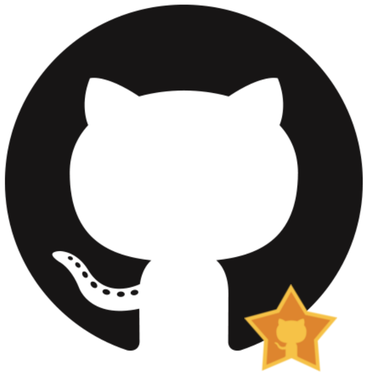
AST оптимизации операций с int

```
>>> import dis

>>> def func():
...     return 1 + 2

>>> dis.dis(func)
  1           RESUME                   0

  2           RETURN_CONST             1 (3)
```



github.com/sobolevn



boosty.to/sobolevn



discord.python.ru